



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ЛИПЕЦКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Институт (факультет)
Кафедра

Компьютерных наук
Прикладной математики и системного анализа

ЛАБОРАТОРНАЯ РАБОТА №2

По дисциплине: «Современные платформы разработки ПО».
На тему: «Системы сборки. Использование зависимостей».

Студент

ПМ-24-1
группа

подпись, дата

Сараев А.П.
фамилия, инициалы

Руководитель

к.т.н.
ученая степень, ученое звание

подпись, дата

Немец С.Ю.
фамилия, инициалы

Липецк 2026

Цель работы

Расширение функциональности программы для работы с пользовательскими объектами данных путем добавления полей различных типов (дата, время, изображение, перечисление), реализации поиска через Stream API, сборки через Gradle/Maven, логирования через Slf4j/Logback и хранения данных в CSV/JSON.

Задание

Необходимо расширить функциональность программы из лабораторной работы №1. В пользовательский объект данных (по вариантам) необходимо добавить дополнительные поля

следующих типов:

- дата;
- время;
- изображение;
- перечисление.

Добавить поля в меню добавления и редактирования данных. Для добавления/редактирования изображения используйте ввод имени файла.

Реализовать поиск объекта по названию/ФИО из загруженного массива. Для поиска использовать Stream API.

Сборка проекта должна выполняться использованием Gradle или Maven.

Логирование необходимо выполнять через библиотеки Slf4j, logback.

Для хранения данных можно использовать один из форматов: CSV или Json.

Также необходимо разработать механизм хранения дополнительных полей в файле данных.

Текст программы

Main.java

```
package org.example;  
  
import org.slf4j.Logger;  
import org.slf4j.LoggerFactory;  
import org.example.models.Laptop;  
import org.example.models.LaptopType;  
import org.example.models.User;
```

```

import org.example.services.AuthService;
import org.example.services.LaptopService;
import org.example.utils.FileManager;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.format.DateTimeParseException;
import java.util.*;

public class Main {
    private static final Logger logger =
        LoggerFactory.getLogger(Main.class);

    private static boolean printLaptops(String username, LaptopService laptopService) {
        List<Laptop> userLaptops = laptopService.getUserLaptops(username);
        if (userLaptops.isEmpty()) {
            System.out.println("You have 0 laptops");
            return false;
        }
        for (int i = 0; i < userLaptops.size(); i++) {
            System.out.println((i + 1) + ". " + userLaptops.get(i));
        }
        return true;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        FileManager fileManager = new FileManager();
        Map<String, User> users = fileManager.loadUsers("users.json");
        List<Laptop> laptops = fileManager.loadLaptops("laptops.json");
        AuthService authService = new AuthService(users);
        LaptopService laptopService = new LaptopService(laptops);

        System.out.print("Username: ");
        String username = sc.nextLine();
        System.out.print("Password: ");
        String password = sc.nextLine();

        if (!authService.login(username, password)) {
            System.out.println("Authentication error");
            return;
        }

        while (true) {
            int choice;
            while (true) {
                try {
                    System.out.println("\nChoose:");
                    System.out.println("1 - Show laptops");
                    System.out.println("2 - Add laptop");
                    System.out.println("3 - Delete laptop");
                    System.out.println("4 - Edit laptop");
                    System.out.println("5 - Find laptop by model");
                    System.out.println("0 - Exit");

                    choice = Integer.parseInt(sc.nextLine());
                    break;
                } catch (NumberFormatException e) {
                    logger.error("Invalid option", e);
                }
            }
        }
    }
}

```

```

}

switch (choice) {
    case 1:
        printLaptops(username, laptopService);
        break;
    case 2:
        try {
            System.out.print("Model: ");
            String model = sc.nextLine();
            System.out.print("Specifications: ");
            String specs = sc.nextLine();
            System.out.print("Price: ");
            double price = Double.parseDouble(sc.nextLine());
            System.out.print("Enter date (YYYY-MM-DD): ");
            LocalDate date;
            try {
                date = LocalDate.parse(sc.nextLine());
            } catch (DateTimeParseException e) {
                logger.error("Error while parsing date");
                break;
            }

            System.out.print("Enter time (HH:MM): ");
            LocalTime time = LocalTime.parse(sc.nextLine());

            System.out.print("Enter image file name: ");
            String path = sc.nextLine();
            String image = fileManager.encodeImage(path);
            if (image.isEmpty()) {
                break;
            }

            System.out.print("Enter type (OFFICE, GAMING, ULTRABOOK): ");
            LaptopType type = LaptopType.valueOf(sc.nextLine());

            laptopService.addLaptop(username, model, specs,
                                    price, date, time, image, type);

        } catch (NumberFormatException e) {
            logger.error("Invalid price", e);
            System.out.println("Price must be number");
        } catch (Exception e) {
            logger.error("Invalid input");
        }
        break;
    case 3:
        if(!printLaptops(username, laptopService)) {
            break;
        }
        System.out.print("Enter laptop number to delete: ");
        try {
            int index = Integer.parseInt(sc.nextLine()) - 1;
            if (!laptopService.deleteLaptop(username, index)) {
                System.out.println("Laptop with this number was not found");
            } else {
                System.out.println("Laptop deleted");
            }
        } catch (NumberFormatException e) {
            logger.error("Invalid input", e);
            System.out.println("Invalid input");
        }
}

```

```

        break;
    case 4:
        if(!printLaptops(username, laptopService)) {
            break;
        }
        System.out.print("Enter laptop number to edit: ");
        try {
            int index = Integer.parseInt(sc.nextLine()) - 1;
            System.out.print("Model: ");
            String model = sc.nextLine();
            System.out.print("Specifications: ");
            String specs = sc.nextLine();
            System.out.print("Price: ");
            double price = Double.parseDouble(sc.nextLine());
            System.out.print("Enter date (YYYY-MM-DD): ");
            LocalDate date = LocalDate.parse(sc.nextLine());

            System.out.print("Enter time (HH:MM): ");
            LocalTime time = LocalTime.parse(sc.nextLine());

            System.out.print("Enter image file name: ");
            String path = sc.nextLine();
            String image = fileManager.encodeImage(path);

            System.out.print("Enter type (OFFICE, GAMING, ULTRABOOK): ");
            LaptopType type = LaptopType.valueOf(sc.nextLine());

            if (!laptopService.editLaptop(username, index, model, specs, price, date, time, image, type)) {
                System.out.println("Laptop with this number was not found");
            } else {
                System.out.println("Laptop updated");
            }
        } catch (NumberFormatException e) {
            logger.error("Invalid input", e);
            System.out.println("Invalid input");
        } catch (Exception e) {
            logger.error("Invalid input", e);
        }
        break;
    case 5:
        System.out.print("Model: ");
        String model = sc.nextLine();
        try {
            Laptop laptop = laptopService.getLaptopByModel(username, model);
            System.out.println(laptop);
        } catch (Exception e) {
            logger.error("Error while finding laptop", e);
        }
        break;
    case 0:
        fileManager.saveLaptops(laptopService.getLaptops(), "laptops.json");
        System.out.println("Data saved");
        return;
    default:
        System.out.println("Invalid option");
    }
}
}
}
}

```

LaptopType.java

```
package org.example.models;

public enum LaptopType {
    OFFICE,
    GAMING,
    ULTRABOOK
}
```

Laptop.java

```
package org.example.models;

import lombok.EqualsAndHashCode;
import lombok.Getter;
import lombok.Setter;

import java.time.LocalDate;
import java.time.LocalTime;

@Getter
@Setter
@EqualsAndHashCode
public class Laptop {

    private String owner;
    private String model;
    private String specs;
    private double price;

    private LocalDate date;
    private LocalTime time;
    private String image;
    private LaptopType type;

    public Laptop() {}

    public Laptop(String owner, String model, String specs, double price,
                  LocalDate date, LocalTime time, String image, LaptopType type) {

        this.owner = owner;
        this.model = model;
        this.specs = specs;
        this.price = price;
        this.date = date;
        this.time = time;
        this.image = image;
        this.type = type;
    }

    @Override
    public String toString() {
        return "Model: " + model +
            ", Specs: " + specs +
            ", Price: " + price +
            ", Date: " + date +
            ", Time: " + time +
            //", Image: " + image +
            ", Type: " + type;
    }
}
```

```
}  
}
```

User.java

```
package org.example.models;  
  
import lombok.Getter;  
import lombok.Setter;  
  
@Getter  
@Setter  
public class User {  
    private String username;  
    private String password;  
  
    public User(String username, String password) {  
        this.username = username;  
        this.password = password;  
    }  
  
    public String toFileString() {  
        return username + ";" + password;  
    }  
}
```

AuthService.java

```
package org.example.services;  
  
import org.slf4j.Logger;  
import org.slf4j.LoggerFactory;  
import org.example.models.User;  
  
import java.util.Map;  
  
public class AuthService {  
    private Map<String, User> users;  
    private static final Logger logger =  
        LoggerFactory.getLogger(AuthService.class);  
  
    public AuthService(Map<String, User> users) {  
        this.users = users;  
    }  
  
    public boolean login(String username, String password) {  
        User user = users.get(username);  
        if (user == null)  
            return false;  
        return user.getPassword().equals(password);  
    }  
}
```

LaptopService.java

```

package org.example.services;

import lombok.Getter;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.example.models.Laptop;
import org.example.models.LaptopType;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
import java.util.NoSuchElementException;

@Getter
public class LaptopService {
    private List<Laptop> laptops;
    private static final Logger logger =
        LoggerFactory.getLogger(LaptopService.class);

    public LaptopService(List<Laptop> laptops) {
        this.laptops = new ArrayList<>(laptops);
    }

    public List<Laptop> getUserLaptops(String username) {
        return laptops.stream()
            .filter(l -> l.getOwner()
                .equals(username)).toList();
    }

    public Laptop getLaptopByModel(String username, String model) {
        return laptops.stream()
            .filter(l -> l.getOwner().equals(username))
            .filter(l -> l.getModel().equals(model))
            .findFirst()
            .orElse(null);
        //orElseThrow(() -> new NoSuchElementException("No laptop with this model"));
    }

    public boolean addLaptop(String user, String model, String specs, double price,
        LocalDate date, LocalDateTime time, String image, LaptopType type) {
        if (model.isEmpty()) {
            logger.error("Model name can't be empty string");
            System.out.println("Model name can't be empty string");
            return false;
        }
        if (specs.isEmpty()) {
            logger.error("Specifications can't be empty string");
            System.out.println("Specifications can't be empty string");
            return false;
        }
        if (price <= 0) {
            logger.error("Invalid price: " + price);
            System.out.println("Price must be bigger than 0");
            return false;
        }
        laptops.add(new Laptop(user, model, specs, price, date, time, image, type));
        return true;
    }

    public boolean deleteLaptop(String user, int index) {
        List<Laptop> userLaptops = getUserLaptops(user);
    }

```



```

    if (index < 0 || index >= userLaptops.size()) {
        return false;
    }
    Laptop laptopToDelete = userLaptops.get(index);
    laptops.remove(laptopToDelete);
    return true;
}

public boolean editLaptop(String user, int index, String model, String specs, double price,
    LocalDate date, LocalTime time, String image, LaptopType type) {
    List<Laptop> userLaptops = getUserLaptops(user);
    if (index < 0 || index >= userLaptops.size()) {
        return false;
    }
    if (model.isEmpty()) {
        logger.error("Model name can't be empty string");
        System.out.println("Model name can't be empty string");
        return false;
    }
    if (specs.isEmpty()) {
        logger.error("Specifications can't be empty string");
        System.out.println("Specifications can't be empty string");
        return false;
    }
    if (price <= 0) {
        logger.error("Invalid price: " + price);
        System.out.println("Price must be bigger than 0");
        return false;
    }
    Laptop laptop = userLaptops.get(index);
    laptop.setModel(model);
    laptop.setSpecs(specs);
    laptop.setPrice(price);
    laptop.setDate(date);
    laptop.setTime(time);
    laptop.setImage(image);
    laptop.setType(type);
    return true;
}
}

```

FileManager.java

```

package org.example.utils;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.core.type.TypeReference;
import com.fasterxml.jackson.datatype.jsr310.JavaTimeModule;
import org.example.models.*;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import java.io.*;
import java.util.*;

import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.Base64;

public class FileManager {
    private static final Logger logger =
        LoggerFactory.getLogger(FileManager.class);
}

```

```

public String encodeImage(String path) {
    try {
        byte[] fileContent = Files.readAllBytes(Paths.get(path));
        return Base64.getEncoder().encodeToString(fileContent);
    } catch (Exception e) {
        logger.error("Error encoding image", e);
        return "";
    }
}

public Map<String, User> loadUsers(String filename) {
    Map<String, User> users = new HashMap<>();
    try {
        ObjectMapper mapper = new ObjectMapper();
        List<User> userList = mapper.readValue(
            new File(filename),
            new TypeReference<List<User>>() {}
        );
        for (User u : userList) {
            users.put(u.getUsername(), u);
        }
    } catch (Exception e) {
        logger.error("Error loading users", e);
    }
    return users;
}

public List<Laptop> loadLaptops(String filename) {
    File file = new File(filename);

    if (!file.exists() || file.length() == 0) {
        return new ArrayList<>();
    }

    try {
        ObjectMapper mapper = new ObjectMapper();
        mapper.registerModule(new JavaTimeModule());
        return mapper.readValue(
            file,
            new TypeReference<List<Laptop>>() {}
        );
    } catch (Exception e) {
        logger.error("Error loading laptops", e);
        return new ArrayList<>();
    }
}

public void saveLaptops(List<Laptop> laptops, String filename) {
    try {
        ObjectMapper mapper = new ObjectMapper();
        mapper.registerModule(new JavaTimeModule());
        mapper.writerWithDefaultPrettyPrinter()
            .writeValue(new File(filename), laptops);
    } catch (Exception e) {
        logger.error("Error saving laptops", e);
    }
}
}

```

Результаты программы на тестовых данных

```
Choose:
1 - Show laptops
2 - Add laptop
3 - Delete laptop
4 - Edit laptop
5 - Find laptop by model
0 - Exit
1
1. Model: Lenovo, Specs: i7 13000, Price: 9999.0, Date: 2020-11-03, Time: 12:45, Type: OFFICE
2. Model: msi, Specs: cool proc, Price: 1234.0, Date: 2020-11-23, Time: 11:11, Type: OFFICE
```

Рисунок 1 — вывод всех объектов

```
Model: Honot
Specifications: a5 7000
Price: 1500
Enter date (YYYY-MM-DD): 2024-11-21
Enter time (HH:MM): 14:44
Enter image file name: C:\Users\JonyJon\Documents\Shared\LSTU\4 sem\modern_platforms\lab1\anime-lucky-star-konata-izumi-pack.png
Enter type (OFFICE, GAMING, ULTRABOOK): ULTRABOOK

Choose:
1 - Show laptops
2 - Add laptop
3 - Delete laptop
4 - Edit laptop
5 - Find laptop by model
0 - Exit
1
1. Model: Lenovo, Specs: i7 13000, Price: 9999.0, Date: 2020-11-03, Time: 12:45, Type: OFFICE
2. Model: msi, Specs: cool proc, Price: 1234.0, Date: 2020-11-23, Time: 11:11, Type: OFFICE
3. Model: Honot, Specs: a5 7000, Price: 1500.0, Date: 2024-11-21, Time: 14:44, Type: ULTRABOOK
```

Рисунок 2 — добавление нового объекта

```
[ {
  "owner" : "admin",
  "model" : "Lenovo",
  "specs" : "i7 13000",
  "price" : 9999.0,
  "date" : [ 2020, 11, 3 ],
  "time" : [ 12, 45 ],
  "image" : "image1",
  "type" : "OFFICE"
}, {
  "owner" : "admin",
  "model" : "msi",
  "specs" : "cool proc",
  "price" : 1234.0,
  "date" : [ 2020, 11, 23 ],
  "time" : [ 11, 11 ],
  "image" : "iVBORw0KGgoAAAANSUgAAAwAAAFHCAYAAAAr9B08AAAACXBIWXMAAAASAAALEgHS3X78AAAgAELEQVR4n0y9e1SU5733p
  ̣/Y0HDjrICCYBQRirU4kKju5QbTxNTJsY4y4ksWu3UbfQNk8jT6qmq30kT9IXfR+zNLVre9h5TbuTFq3apM/2g6LCjDvFUTGV0o0jaqIdlQFRsAoMMspRffp
  ̣+45rrnmvP5vgf4fdaaxRzvue9rBpj53Rp̣
```

Рисунок 3 — json файл для хранения объектов

Вывод

В результате работы программа дополнена новыми полями (дата, время, изображение, перечисление), реализован поиск по имени/ФИО с использованием Stream API, организовано логирование и хранение данных в файловом формате (JSON) с возможностью загрузки и сохранения.